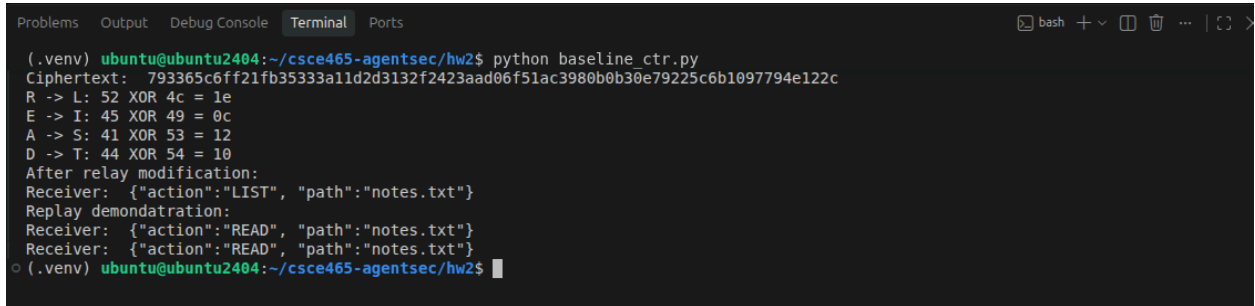


Task 0

```
DH Parameters: (3072 bit)
GROUP: ffdhe3072
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python3 --version
Python 3.12.3
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/ubuntu/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

Task 1

A terminal window with a dark background and light text. The prompt is (.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2\$. The command python baseline_ctr.py has been executed. The output shows a long hex string for the ciphertext, followed by XOR calculations for letters R, E, A, and D. Then, it shows the modification of the ciphertext to change the command from READ to LIST. Finally, it shows the receiver processing the modified ciphertext three times, all interpreting it as a READ command.

```
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$ python baseline_ctr.py
Ciphertext: 793365c6ff21fb35333a11d2d3132f2423aad06f51ac3980b0b30e79225c6b1097794e122c
R -> L: 52 XOR 4c = 1e
E -> I: 45 XOR 49 = 0c
A -> S: 41 XOR 53 = 12
D -> T: 44 XOR 54 = 10
After relay modification:
Receiver: {"action":"LIST", "path":"notes.txt"}
Replay demondatratiom:
Receiver: {"action":"READ", "path":"notes.txt"}
Receiver: {"action":"READ", "path":"notes.txt"}
Receiver: {"action":"READ", "path":"notes.txt"}
(.venv) ubuntu@ubuntu2404:~/csce465-agentsec/hw2$
```

The design of this program is that it encrypts the command `{“action”:“READ”, “path”:“notes.txt”}` using AES in CTR mode. The program does not use a MAC or any other integrity check. There is also a relay function that then changes the encrypted command so that READ would become LIST even though the relay doesn’t know the encryption key. Finally, I implemented the receiver accepting the same ciphertext more than once to test replay behavior.

What happens is that the relay was able to modify the ciphertext so that the receiver decrypted the command as `{“action”:“LIST”, “path”:“notes.txt”}` instead of the original “READ” command. This worked because the relay changed the ciphertext using the XOR difference between the original and the replacement word. The receiver had no way to detect that the ciphertext had been modified. I also sent the original ciphertext to the receiver twice, and the receiver processed the same “READ” command twice.

The root cause is that AES-CTR by itself only provides confidentiality. It doesn’t provide integrity or replay protection. Because CTR mode uses XOR, changing certain ciphertext bits causes predictable changes in the decrypted plaintext. Since there was no MAC, the receiver could not detect the modification. Also, there was no sequence number, nonce-tracking system, or other freshness check, so the receiver couldn’t recognize that the same ciphertext had already been processed.

Task 2

The main part of the design is Diffie-Hellman key establishment. Diffie-Hellman lets the gateway and the node come up with the same shared secret without directly sending that secret to each other. Each side makes a private DH value and a public value. They exchange the public values, and then both sides are able to calculate the same shared secret. And for every new session, the program makes new temporary DH values.

The design also uses RSA-PSS digital signatures. The digital signatures are used so the gateway and node can prove who they are. DH by itself can make a shared secret, but it doesn't prove who is on the other side. The gateway signs the handshake information with its RSA private key, and the node does the same thing. Then each side checks the other side's signature during the correct RSA public key. The handshake transcript includes both identities, both DH public values and both random nonces. The role is also included when the signature is made. Signing all of this information is important because it connects everything together. If an attacker changes one of the identities, changes a nonce, or replaces one of the DH public values, the transcript hash changes. The old signature will no longer match, so the handshake should be rejected. The main purpose of the nonces is to add freshness to the handshake. This helps with replay resistance. Since a new handshake uses new nonces, an old handshake should not match the new session. This makes it harder for an attacker to reuse old handshake messages.

The design also gives forward secrecy because the DH private values are temporary. The RSA keys are long-term keys, but they are only used for signing. They are not used to create the shared secret. Because of this, if someone steals an RSA private key later, they should not automatically be able to figure out the old session secrets.

Key separation is another important part of the design. After the shared secret is created, the program doesn't use the exact same key for everything. It creates different keys for gateway-to-node encryption, gateway-to-node MAC, node-to-gateway encryption, node-to-gateway MAC, and the session ID. This keeps one key from being reused for multiple jobs, which is overall safer.

In a man-in-the-middle attacks, an attacker tries to sit between the gateway and the node. If there were no signatures, the attacker could replace the DH public values with their own. The attacker could then make one shared secret with the gateway and another shared secret with the node. The gateway and

node might think they are talking to each other, but the attacker would be in the middle. With signatures, this attack is much harder because the attacker would need to create a valid RSA signature for the changed DH values, which they should not be able to do due to not having the real private key. The role values help stop reflection attacks, where an attacker takes a valid message from one side and sends it back to that same side as if it came from the other party. Since the gateway signs using the gateway role and the node signs using the node role, the signatures are different even if the rest of the handshake information is the same. A gateway signature should not be accepted as a node signature.

If the signatures were completely removed, the Diffie-Hellman math would still work, but the connection would no longer be authenticated. The gateway and node could still make shared secrets, but they would not know for sure who they made the secret with. This would make a basic man-in-the-middle attack possible. An attacker could replace the public values, create separate keys with each side, read the messages, change them, and then send them on. Overall, the Diffie-Hellman part gives the shared secret, but the signatures are what prove who the shared secret is being made with.

Task 3

For this task, I implemented a secure record layer that protects messages after the handshake creates the session keys. The main goal is to give the messages confidentiality, integrity and replay protection. I used AES-256 in CTR mode to encrypt the plaintext so someone watching the communication cannot easily read the message. Since AES-CTR by itself doesn't protect against someone changing the ciphertext, I also used HMAC-SHA-256 to authenticate the header, IV, and ciphertext. The MAC is checked before the message is decrypted, which prevents modified records from being accepted. Each record also contains a version number, direction, sequence number, message type, and ciphertext length. The sequence number starts at 0 and must match the exact next expected value, so an attacker cannot simply replay an old valid message or send messages out of order. The IV is created using the 8-byte session ID combined with the 8-byte sequence number, which makes a different IV for every record in the session and helps prevent IV reuse with AES-CTR. I also used separate encryption and MAC keys for sending and receiving, along with separate direction values. This provides key separation and prevents a message going one direction from being treated as a valid message going the opposite direction.

Secure note

The attacker model for this secure record layer assumes that an attacker can see messages being sent, copy them, replay old messages, or modify part of them. The attacker could replace the message with a different one. However, the attacker doesn't know the secret encryption key or MAC key. The goal of the record layer is to keep the message private, make sure it was not changed, and stop old messages from being replayed. Some things are not handled by the record layer.

CTR mode needs a unique IV every time a message is encrypted with the same key. CTR works by creating a stream of random-looking bytes using the key and IV. The plaintext is then XORed with this stream to create the cipher text. If the same IV is used twice with the same key, the same stream is created again. This could be harmful because an attacker can compare the two ciphertexts and learn information about the original plaintexts. IVs should never repeat when using the same encryption key.

It's very important to verify the MAC before decrypting the ciphertext. The MAC is used to check that the message was not changed. If the MAC is wrong, the receiver should reject the record immediately. If the receiver decrypts the message first, it is processing data that may have been changed by an attacker. Checking the MAC first helps prevent cases, such as error messages or behaviors that cause information to leak, and makes sure only valid records are decrypted.

The encryption key and MAC key should also be different. The encryption key is used to hide the plaintext, while the MAC key is used to check if the record was changed. Since these keys have different jobs, using separate keys is safer. Reusing the same key for different cryptographic operations can sometimes cause security problems. A key derivation function can take one shared secret from the handshake and create separate keys for encryption and authentication. This can be compared to AES-GCM. AES-GCM is an AEAD mode, which means it handles encryption and authentication together. With CTR and HMAC, the programmer has to separately encrypt the message and calculate a MAC. Both methods can be secure if they are used correctly. AES-GCM is usually easier because encryption and authentication are built into one mode. However, AES-GCM still requires unique nonces, just like CTR requires unique IVs. Finally, a valid protected record does not automatically mean the command inside the record is safe. For example, a record could have a correct MAC and

contain a real command asking a tool to delete a file or access private information. The cryptography only proves that the record was not changed and came from someone with the correct key. The application still needs authorization checks before performing the tool call. This means that encryption and authentication protect the message, while authorization protects what the system is actually allowed to do.